

《面向对象技术与方法》结课设计

ClauseMind 项目管理文档

学生姓名：付翔宇

项目类型：个人独立开发项目

系统类型：前后端分离的智能化 Web 应用

提交日期：2026 年 6 月 21 日

目录

说明：若打开 Word 后目录未自动显示，请在 Word 或 LibreOffice 中选择“更新目录/更新域”。

目录将在打开文档后生成

项目管理文档

ClauseMind 合同智能审查系统 —— 项目管理文档

1. 项目组成员及分工

△ 说明：本项目为个人独立开发项目

成员	角色	承担工作	贡献占比
付翔宇	全栈开发	系统架构设计、后端开发 (FastAPI)、前端开发 (React)、文档编写、测试	100%

1.1 具体分工详情

模块	负责内容	技术栈
系统架构	项目整体架构设计、技术选型、数据库设计	FastAPI + React + SQLite
后端 - 用户系统	User 模型、注册/登录/鉴权 API、JWT	FastAPI + JOSE + Bcrypt
后端 - 合同管理	Contract 模型、上传/列表/详情/删除	FastAPI + SQLAlchemy
后端 - 文档解析	MinerU 集成、解析状态管理、错误处理	subprocess + MinerU CLI
后端 - 标准化	文档结构提取、正则解析、JSON 结构化	Python + regex
后端 - Agent 框架	BaseAgent、LLMClient、JSON 解析重试	httpx + OpenAI-compatible API
后端 - 多 Agent	6 个专用 Agent 实现、工作流编排	异步 + 顺序执行
后端 - 审查服务	ReviewService、结果持久化、事务管理	SQLAlchemy + 事务
后端 - 报告服务	报告生成、Markdown/HTML 导出	Python + Markdown
后端 - 管理后台	用户/合同/任务/日志/统计 API	FastAPI + SQLAlchemy
后端 - 测试	单元测试、集成测试、API 测试	Pytest + HTTPX
前端 - 认证	登录/注册页面、Token 管理、路由守卫	React + Ant Design + Zustand
前端 - 合同管理	上传/列表/详情页面	React + Ant Design Table
前端 - 解析展示	解析状态、Markdown 预览、标准化结果	React + react-markdown
前端 - 审查展示	步骤条、Agent 日志、风险/建	React + Ant Design Steps

	议/报告	
前端 - 管理后台	管理员表格、筛选、统计展示	React + Ant Design + ECharts
文档	需求文档、设计文档、API 文档、数据库文档	Markdown + Mermaid

2. AI 应用情况报告

2.1 使用的 AI 工具

AI 工具	用途	使用比例
GitHub Copilot	代码生成、代码补全、架构设计、Bug 修复	70%
ChatGPT / DeepSeek	技术方案讨论、代码 Review、优化建议	20%
LLM API (审查 Agent)	系统运行时调用，分析合同文本	10%

2.2 AI 在开发各阶段的应用

(1) 需求分析阶段

- 使用 AI 分析课程设计要求，提取关键点
- AI 辅助梳理功能清单和系统边界
- AI 生成竞品分析初稿，人工审核后定稿

(2) 系统设计阶段

- AI 协助进行技术选型分析 (FastAPI vs Spring Boot)
- AI 生成 UML 图 (类图、时序图、组件图)
- AI 辅助数据库表结构设计
- AI 生成 API 接口定义

(3) 编码实现阶段

- 后端核心代码：Agent 框架、LLM Client、Normalizer 等核心逻辑由 AI 辅助生成
- 前端页面：90% 的 React 组件代码由 Copilot 生成
- **API** 路由：AI 根据接口定义自动生成 FastAPI 路由代码
- 测试代码：AI 生成 pytest 测试用例
- 数据库迁移：AI 辅助编写 Alembic migration 脚本

(4) 调试测试阶段

- AI 帮助分析错误日志，提供修复建议
- AI 辅助编写测试数据 (seed_demo.py)
- AI 协助排查 bcrypt 兼容性等环境问题

(5) 文档编写阶段

- AI 生成需求文档、设计文档、API 文档的初稿
- 人工核对文档内容与实现的一致性
- AI 辅助生成 Mermaid 图表代码

2.3 AI 应用经验总结

方面	经验
有效做法	1. 将复杂任务拆分为小步骤，逐步让 AI 实现 2. 先定义好接口和数据结构，再让 AI 填充实现 3. 让 AI 先生成测试，再根据测试实现功能
注意事项	1. AI 生成的代码需要人工审查，可能存在逻辑错误 2. AI 可能使用过时的 API 或库版本 3. AI 对项目全局理解有限，需要人工把控架构
改进空间	1. 可以使用更结构化的 prompt 模板 2. 建立项目规范文档让 AI 保持一致风格 3. 积累常见问题的解决方案库

3. 开发记录

3.1 开发时间线

日期	阶段	完成工作	产出文件
2026-06-13	Phase 0	项目初始化、需求分析	CLAUDE.md、README.md
2026-06-14	Phase 1	项目骨架搭建、目录结构、配置	frontend/、backend /、 docs/
2026-06-15	Phase 2	用户认证模块实现	models/ user.py、api/v1/ auth.py
2026-06-15	Phase 3	合同管理功能实现	models/ contract.py、api/ v1/contracts.py
2026-06-15	Phase 4	MinerU 文档解析集成	mineru_service.py、api/v1/parse.py
2026-06-15	Phase 5	文档标准化实现	document_normaliz er.py
2026-06-15	Phase 6	首个 Agent (Profile)	base_agent.py、con tract_profile_agent.p y
2026-06-15	Phase 7	完整多 Agent 工作流	6 个 Agent、orchestrator .py、review_service.

			py
2026-06-15	Phase 8	审查结果展示页面	风险/建议/报告前端页面
2026-06-15	Phase 9	管理员后台与文档	admin.py、docs/*.md
2026-06-16	优化	GPU 配置、测试隔离、文档完善	环境配置优化
2026-06-16	Phase 2 完善	完整认证模块（前端连接）	authStore、LoginPage、RegisterPage
2026-06-21	最终	课程设计文档完善	需求/设计/项目管理文档、UML 图

3.2 Git 提交记录

提交时间	提交信息	分支
2026-06-15	first commit	main
2026-06-15	feat: scaffold initial ClauseMind architecture	main
2026-06-15	feat(Phase2): implement user auth module	main
2026-06-15	fix(Phase3): remove auto create_all, use Alembic-only for migrations	main
2026-06-15	feat(Phase3): implement contract upload and management	main
2026-06-15	feat(Phase4): implement MinerU parse integration	main
2026-06-15	chore: vendor MinerU source	main
2026-06-15	fix(Phase4)+feat(Phase5): remove server paths from parse-result + normalization	main
2026-06-15	feat(Phase6): implement first contract profile agent	main
2026-06-15	feat(Phase7): implement full multi-agent review workflow	main
2026-06-15	feat(Phase7): connect ReportPage to real review report API	main
2026-06-15	feat(Phase8): complete review result pages	main
2026-06-15	feat(Phase9): add admin dashboard and deliverable docs	main

2026-06-15	feat: add final admin exports demo data and docs	main
2026-06-15	docs: update project README	main
2026-06-16	fix: route MinerU subprocess to configured GPU	main
2026-06-16	test: isolate pytest database from dev data	main
2026-06-16	docs: remove machine-specific paths from config docs	main

3.3 分支管理

分支	用途
main	主分支，所有功能直接提交或合并到 main

3.4 主要开发活动详情

重点活动 1：多 Agent 框架实现

活动描述：设计并实现了 BaseAgent 抽象基类，包含 LLM 调用、JSON 解析、重试机制、提示词模板加载等通用功能。在此基础上实现了 6 个专用 Agent。

关键决策：

- 使用 async/await 实现 Agent 的异步执行
- JSON 解析失败时自动重试一次（携带更明确的指令）
- 每个 Agent 有独立的提示词模板文件（prompts/*.txt）

重点活动 2：MinerU 集成

活动描述：实现 MinerUService 通过 subprocess 调用 MinerU CLI，收集 Markdown、JSON、Layout PDF 等输出文件。

技术难点：

- MinerU 输出路径不固定，需要扫描目录自动发现
- 超时和错误处理需要包装为用户友好的中文提示
- GPU 环境配置需要透传 CUDA_VISIBLE_DEVICES

重点活动 3：前端全流程实现

活动描述：实现了从登录→上传→解析→审查→报告的完整前端流程，包含 12 个页面、8 个可复用组件、5 个 API 模块。

关键设计：

- Zustand 管理认证状态，localStorage 持久化
- Ant Design Steps 组件展示 Agent 审查进度
- react-markdown 渲染审查报告
- 路由守卫确保未登录用户无法进入受保护页面

4. 项目总结（可选）

4.1 成果回顾

ClauseMind 合同智能审查系统从零开始，在约一周的开发周期内完成了一个功能完整的、基于 React + FastAPI + MinerU + 多 Agent 的智能化系统。系统实现了：

- 后端：10 个数据表、20+ REST API、6 个 AI Agent 工作流、完整的用户认证和权限控制
- 前端：12 个功能页面、8 个可复用组件、前后端分离架构
- 文档：需求/设计/API/数据库/UML 共 5 份文档
- 测试：15 个测试文件，覆盖核心模块
- 总代码量：约 7,000 行

4.2 技术亮点

1. 多 **Agent** 协作架构：6 个专用 Agent 顺序执行，分工明确，可独立调试和演进
2. 端到端 **AI** 集成：从文档解析（MinerU）到 LLM 分析（Agent），全流程 AI 驱动
3. 工程完整性：Alembic 迁移管理、Pydantic 数据校验、JWT 鉴权、单元测试
4. 模块化设计：核心功能模块清晰分离，便于扩展和维护

4.3 不足与改进方向

5. **LLM Agent** 输出稳定性：LLM 输出的 JSON 格式偶尔不符合预期，需要更强的提示词工程
6. 异步任务管理：当前 Agent 审查是同步阻塞的，可以改为 BackgroundTasks 或 Celery
7. 前端测试：缺少前端单元测试和 E2E 测试
8. 错误处理：部分边界情况（如超大文件、网络断开）的处理可以更优雅
9. 多语言支持：当前仅支持中文合同，可扩展支持英文和双语合同